

Sección 5-1: Funciones de Conversión

Objetivos

- En esta lección se abordan los siguientes objetivos:
 - Proporcionar un ejemplo de conversión de tipo de dato implícita y explícita y una conversión de tipo de dato implícita.
 - Explicar el motivo de la importancia, desde una perspectiva de negocio, de que un lenguaje tenga capacidades de conversión de datos incorporadas.
 - Crear una consulta SQL que aplique correctamente las funciones de una sola fila TO_CHAR, TO_NUMBER y TO_DATE para obtener el resultado deseado.

Objetivos

- Aplicar el modelo de formato de fecha y/o caracteres adecuado para producir una salida deseada.
- Explicar y aplicar el uso de YY y RR para devolver el año correcto como se almacenó en la base de datos.

Objetivo

- Imagine que tuviera que leer todos los libros de texto en archivos de texto sin párrafos y sin mayúsculas.
 - Sería difícil leerlos.
 - Afortunadamente, existen programas de software disponibles que permiten poner en mayúsculas y aplicar color al texto, subrayarlos, ponerlos en negrita, centrarlo y agregar gráficos.
- En el caso de bases de datos, los cambios de formato y visualización se realizan utilizando funciones de conversión.
 - Estas funciones permiten mostrar los números en la moneda local, aplicar una gran variedad de formatos a las fechas, mostrar la hora incluyendo hasta los segundos y hacer un seguimiento del siglo al que hace referencia una fecha.

Tipos de Dato

- Cuando se crea una tabla para una base de datos, el programador SQL debe definir qué tipo de dato se almacenará en cada uno de los campos de la tabla.
- En SQL, hay diferentes tipos de dato. Estos tipos de dato definen el dominio de valores que puede incluir cada columna.
- Para esta lección, utilizará:
 - VARCHAR2
 - CHAR
 - NUMBER

- DATE

Descripción de Tipos de Dato

- **VARCHAR2**: se utiliza para datos de caracteres de longitud variable, incluidos los números, los guiones y los caracteres especiales.
- **CHAR**: se utiliza para datos de texto y de caracteres de longitud fija, incluidos los números, los guiones y los caracteres especiales.

Descripción de Tipos de Dato

- **NUMBER**: se utiliza para almacenar datos numéricos de longitud variable. No se permiten guiones, texto u otros datos no numéricos. La moneda se almacena como tipo de dato numérico.
- **DATE**: se utiliza para valores de fecha y hora. Internamente, Oracle almacena las fechas como números y, por defecto, se muestra información de DATE como DD-Mes-YYYY (por ejemplo, 23-Oct-2013).

Conversión de Tipo

- Oracle Server puede convertir automáticamente datos VARCHAR2 y CHAR en tipos de dato NUMBER y DATE.
- Puede convertir datos de tipo NUMBER y DATE de nuevo en un tipo de dato CHARACTER.
- A esto se le conoce como conversión de datos implícita.

Conversión de Tipo

- Aunque esto es una característica útil, siempre es mejor realizar conversiones de tipo de dato explícitas para garantizar la fiabilidad en las sentencias SQL.

Conversión de Tipo

- Las cuatro funciones de conversión de tipos de dato que aprenderá son:
 - Convertir un tipo de dato de fecha en tipo de dato de carácter.
 - Convertir un tipo de dato numérico en tipo de dato de carácter.

Conversión de Datos de Fecha en Datos de Caracteres

- Por lo general, es aconsejable que un formato de fecha se convierta de su formato DD-Mes-YYYY por defecto en otro formato especificado por el usuario.
- La función que permite llevar esto a cabo es: `TO_CHAR(date_column_name, 'format model you specify')`.
- El 'modelo de formato' debe estar entre comillas simples y es sensible a mayúsculas/minúsculas.
- Separe el valor de fecha del modelo de formato con una coma.

Conversión de Datos de Fecha en Datos de Caracteres

- Se puede incluir cualquier elemento de formato de fecha válido.
 - Utilice sp para deletrear un número.
 - Utilice th para que el número aparezca como un ordinal (1º, 2º, 3º y así sucesivamente).
 - Utilice un elemento fm para eliminar los espacios en blanco o eliminar los ceros iniciales de la salida.

Conversión de Datos de Fecha en Datos de Caracteres

- Las tablas muestran los diferentes modelos de formato que se pueden utilizar.
 - YYYY: Año completo en números.
 - YEAR: Año en letra.
 - MM: Valor de dos dígitos del mes.
 - MONTH: Nombre completo del mes.
 - MON: Abreviatura de tres letras del mes.
 - DY: Abreviatura de tres letras del día de la semana.
 - DAY: Nombre completo del día de la semana.
 - DD: Día numérico del mes.

Conversión de Datos Numéricos en Datos de Caracteres (VARCHAR2)

- Los números almacenados en la base de datos no tienen formato.
 - Esto significa que no tienen ningún signo/símbolo de moneda, comas, decimales, ni ningún otro formato.
 - Para agregar formato, en primer lugar debe convertir el número en un formato de caracteres.
- La función SQL que se utiliza para convertir un número en un formato de carácter deseado es: `TO_CHAR(number, 'format model')`.

Conversión de Caracteres en Números

- Por lo general, es aconsejable convertir una cadena de caracteres en un número. La función necesaria para realizar esta conversión es: `TO_NUMBER(character_string, 'format model')`.
- El modelo de formato es opcional, pero se debería incluir si la cadena de caracteres que se va a convertir contiene cualquier carácter que no sean números.

Conversión de Caracteres en Fechas

- Para convertir una cadena de caracteres en un formato de fecha, utilice: `TO_DATE('character_string', 'format model')`.
- Esta conversión toma una cadena de caracteres que no sea un valor de fecha como, por ejemplo, "3 noviembre de 2001" y lo convierte en un valor de fecha.

Reglas del Modificador fx

- Cuando realiza una conversión de caracteres en fecha, el modificador fx (formato exacto) especifica la coincidencia exacta entre el argumento de carácter y el modelo de formato de fecha.
 - La puntuación y el texto entre comillas en el argumento de caracteres deben coincidir con las partes correspondientes del modelo de formato exactamente (excepto en lo que respecta a mayúsculas/minúsculas).
 - El argumento de carácter no puede tener espacios en blanco adicionales.

Formato de Fecha RR y Formato de Fecha YY

- Todos los datos de fecha ahora se almacenan utilizando años de cuatro dígitos (YYYY).
- Si los datos que se van a convertir de datos de caracteres a datos de fecha solo contienen un año de dos dígitos, Oracle tiene una forma de interpretar estas fechas con el siglo correcto.
- Si YY se utiliza en el modelo de formato, se asume que el año pertenece al siglo actual.
- Si el año de dos dígitos no está en el siglo actual, utilizamos RR.

Algunas Sencillas Reglas

- Si el formato de fecha se especifica con el formato RR, el valor de retorno tiene dos posibilidades, en función del año actual.
 - Si el año actual está entre 00-49:
 - Fechas a partir de 0-49: la fecha estará en el siglo actual.
 - Fechas a partir de 50-99: la fecha estará en el siglo pasado.
 - Si el año actual está entre 50-99:
 - Fechas a partir de 0-49: la fecha estará en el siglo siguiente.
 - Fechas a partir de 50-99: la fecha estará en el siglo actual.

Terminología

- Entre los términos clave utilizados en esta lección se incluyen:
 - CHAR
 - DATE
 - Formato de fecha DD
 - Funciones de conversión
 - fm
 - NUMBER
 - Formato de fecha RR
 - TO_CHAR
 - TO_DATE
 - TO_NUMBER
 - VARCHAR2
 - Modificador fx

Resumen

- En esta lección, debe haber aprendido lo siguiente:
 - Proporcionar un ejemplo de conversión de tipo de dato implícita y explícita y una conversión de tipo de dato implícita.
 - Explicar el motivo de la importancia, desde una perspectiva de negocio, de que un lenguaje tenga capacidades de conversión de datos incorporadas.
 - Crear una consulta SQL que aplique correctamente las funciones de una sola fila TO_CHAR, TO_NUMBER y TO_DATE para obtener el resultado deseado.
 - Aplicar el modelo de formato de fecha y/o caracteres adecuado para producir una salida deseada.
 - Explicar y aplicar el uso de YY y RR para devolver el año correcto como se almacenó en la base de datos.
-

5-2 Funciones NULL

Objetivos

- En esta lección se abordan los siguientes objetivos:
 - Mostrar y explicar la evaluación de una función anidada.
 - Enumerar al menos cuatro funciones generales que funcionan con cualquier tipo de dato y están relacionadas con el tratamiento de valores nulos.
 - Explicar el uso de las funciones COALESCE y NVL.
 - Explicar el uso de funciones generales para tratar con valores nulos en los datos.
 - Crear y ejecutar una consulta SQL que aplica correctamente las funciones NVL, NVL2, NULLIF y COALESCE de una sola fila.
-

Objetivo

- SQL utiliza un conjunto de funciones generales diseñadas específicamente para tratar valores nulos.
 - El manejo de valores nulos afecta a la forma en que las expresiones se evalúan, los cálculos y el orden de los datos.
-

Método de Evaluación de las Funciones

- Las funciones anidadas se evalúan desde el nivel más profundo al menos profundo.

- Ejemplo:

Unset

```
SELECT TO_CHAR(NEXT_DAY(ADD_MONTHS(hire_date, 6), 'FRIDAY'),  
'fmDay, Month ddth, YYYY') AS "Next Evaluation"  
FROM employees  
WHERE employee_id = 100;
```

Funciones Relacionadas con los Valores Nulos

- Las cuatro funciones principales relacionadas con valores nulos son:
 - NVL
 - NVL2
 - NULLIF
 - COALESCE

Función NVL

- Convierte un valor nulo en un valor conocido de un tipo de dato fijo.
- Ejemplo:

Unset

```
SELECT last_name, NVL(commission_pct, 0)  
FROM employees  
WHERE department_id IN (80, 90);
```

Función NVL2

- Evalúa tres expresiones: si la primera no es nula, devuelve la segunda; si es nula, devuelve la tercera.
- Ejemplo:

Unset

```
SELECT last_name, salary,  
NVL2(commission_pct, salary + (salary * commission_pct), salary)  
AS income  
FROM employees  
WHERE department_id IN (80, 90);
```

Función NULLIF

- Compara dos expresiones: si son iguales, devuelve nulo; si no son iguales, devuelve la primera expresión.
- Ejemplo:

Unset

```
SELECT first_name, LENGTH(first_name) AS "Length FN", last_name,  
LENGTH(last_name) AS "Length LN",  
NULLIF(LENGTH(first_name), LENGTH(last_name)) AS "Compare Them"  
FROM employees;
```

Función COALESCE

- Devuelve la primera expresión no nula de una lista.
- Ejemplo:

Unset

```
SELECT last_name, COALESCE(commission_pct, salary, 10) AS "Comm"  
FROM employees  
ORDER BY commission_pct;
```

Terminología

- Entre los términos clave utilizados en esta lección se incluyen:

- NVL
 - NVL2
 - NULLIF
 - COALESCE
-

Resumen

- Se ha aprendido a:
 - Mostrar y explicar la evaluación de funciones anidadas.
 - Utilizar funciones generales relacionadas con valores nulos.
 - Aplicar correctamente las funciones NVL, NVL2, NULLIF y COALESCE en consultas SQL.
-
-

5-3 Expresiones Condicionales

Objetivos

- En esta lección se abordan los siguientes objetivos:
 - Comparar y contrastar las funciones DECODE y CASE.
 - Crear y ejecutar una consulta SQL que utiliza correctamente las funciones DECODE y CASE.
 - Crear y ejecutar dos métodos para implantar la lógica condicional IF-THEN-ELSE.
-

Objetivo

- Poder tomar decisiones es esencial en el modelado de datos.
 - En SQL, este tipo de elecciones implica métodos de procesamiento condicionales como IF-THEN-ELSE.
-

Expresiones Condicionales

- Las dos expresiones condicionales más comunes en SQL son CASE y DECODE.

- CASE es compatible con el estándar ANSI/ISO SQL 99, mientras que DECODE es propiedad de Oracle. Ambas devuelven la misma información pero con sintaxis diferente.
-

Expresión CASE

- Realiza la función de una sentencia IF-THEN-ELSE.
- Ejemplo de sintaxis:

Unset

```
CASE expr WHEN comparison_expr1 THEN return_expr1
WHEN comparison_expr2 THEN return_expr2
ELSE else_expr END
```

- Ejemplo en una consulta:

Unset

```
SELECT last_name,
CASE department_id
WHEN 90 THEN 'Management'
WHEN 80 THEN 'Sales'
WHEN 60 THEN 'It'
ELSE 'Other dept.'
END AS "Department"
FROM employees;
```

Expresión DECODE

- Evalúa una expresión de manera similar a la lógica IF-THEN-ELSE.
- Ejemplo de sintaxis:

Unset

```
DECODE(column|expression, search1, result1 [, search2,  
result2,...,] [, default])
```

- Ejemplo en una consulta:

Unset

```
SELECT last_name,  
DECODE(department_id,  
90, 'Management',  
80, 'Sales',  
60, 'It',  
'Other dept.')  
AS "Department"  
FROM employees;
```

Terminología

- Entre los términos clave utilizados en esta lección se incluyen:
 - CASE
 - Expresión condicional
 - DECODE

Resumen

- En esta lección, se ha aprendido a:
 - Comparar y contrastar las funciones DECODE y CASE.
 - Crear y ejecutar consultas SQL que utilizan correctamente las funciones DECODE y CASE.
 - Aplicar la lógica condicional IF-THEN-ELSE en SQL.
-